

Multiply: Tokens Whose Fees Run a Leveraged Position

multiply.cash
Robinhood Chain

August 2026

Abstract

The system described here is a token launchpad on Robinhood Chain in which every launched token permanently locks the entirety of its liquidity and streams its trading fees into a live leveraged perpetual position on Lighter, an onchain order-book exchange. The token itself is deliberately inert: fixed supply, no administrative authority, no transfer tax. The engine sits outside it. The pool's 1% swap fee is collected by an ownerless locker contract that burns the token side of every claim and forwards the quote side to a per-token treasury, which posts it as collateral on a perpetual market chosen at launch — any Lighter market, long or short, at $2\times$ to $20\times$ leverage. Profit is realized tranche by tranche under a fixed take-profit policy; 75% of realized profit buys back and burns the token, and the remaining 25% accrues to the operator as the protocol's only revenue. This paper specifies the launch mechanics, the fee circuit, the tranche accounting, and the trust model, and states the properties and risks that follow from the design.

1. Introduction

Token launchpads have converged on a common design. A token with fixed supply is sold through a curve or seeded into a pool, liquidity is locked, and the trading fees the token generates are treated as income: split among the creator, the protocol, and sometimes the pool itself. The fee stream is distributed, not deployed.

This paper specifies a launchpad in which the fee stream is working capital. Each token launches with its full supply seeded one-sided into a Uniswap v3 pool quoted in USDG, and the liquidity position is permanently locked in a contract with no owner and no withdrawal path. The locker burns the token side of every fee claim and forwards the quote side to a treasury that exists for that token alone. The treasury posts those fees as collateral on a leveraged perpetual — market, direction, and leverage chosen by the creator at launch — executed on Lighter's Robinhood deployment [1]. Realized profit returns on-chain, where 75% of it buys the token back on its own pool and burns it.

The design differs from reserve-coupled systems, in which a tokenized leveraged position is the reserve asset of the curve and the token's price mechanically tracks the underlying. Here the coupling is deliberately loose. The token trades on an ordinary spot pool; the perpetual affects its price only through realized flows — buybacks — and through supply reduction. Two consequences follow. First, failure of the position cannot cascade into the pool: a liquidation costs the engine its posted collateral and nothing else, and the engine restarts from the next fees collected. Second, exposure to the underlying is realized rather than

marked: holders receive it as burns, not as a continuously repricing reserve.

Sections 2–5 specify the mechanics. Section 6 states the properties the mechanics produce, including an explicit trust model. Section 7 states the risks.

2. Launch Mechanics

Token. A standard ERC-20 with fixed supply $S = 10^9$ units, minted once in the constructor. The contract has no owner, no administrative functions, no transfer tax, no pause, and no subsequent minting: there is nothing to renounce, because authority over the token never exists. A transfer to the conventional dead address is routed to a true burn — total supply decreases, and no phantom balance accumulates — while a transfer to the zero address reverts, so burns are always explicit. The quantity reported as burned is the identity $B(t) = S - \text{supply}(t)$, recomputable by anyone from the chain.

One-sided seeding. At launch the entire supply is placed as a single one-sided concentrated-liquidity position on a canonical Uniswap v3 pool [3] against USDG [4], at fee tier $\varphi = 1\%$. The opening price p_0 is set from a target valuation V_0 (currently \$4,000):

$$p_0 = V_0 / S, \quad \text{so} \quad MC_0 = p_0 \cdot S = V_0$$

Every token opens at the same dollar valuation regardless of market or leverage choice. There is no graduation event: the pool seeded at launch is the token's market for life. Purchases walk up the ask ladder of the locked position; there is no migration, no LP token, and no moment at which liquidity changes hands.

The locker. In the launch transaction the LP NFT is transferred into the locker through the ERC-721 receive hook, which atomically registers an immutable pair: the fee recipient (the token’s treasury) and the burn side (the token itself). Only the position manager can invoke the hook, and a position can never be re-registered. The locker’s single operational function, *collect*, is permissionless: anyone may call it for any position, and its only reachable outcome is that the token side of accrued fees is sent to the dead address and the quote side to the registered treasury. The contract has no owner, no withdraw, no transfer-out, no liquidity removal, and no upgrade path. If one side of the pair is ever frozen, a rescue variant collects the other side alone, to the same fixed destinations.

Per-token treasuries. Each token’s treasury address is derived deterministically from an operator master secret k as $\text{HMAC-SHA256}(k, \text{addr}_{\text{token}})$ [5]. The private key exists only inside the keeper process; only the address is ever persisted. The treasury is simultaneously the locker’s registered fee recipient and the owner of the token’s account on Lighter, and the venue accepts orders for an account only from its owner’s key. Tokens are therefore pairwise isolated — one token’s funds can never touch another token’s position — and the creator, who never holds any of these keys, cannot drain, close, or borrow against the position backing their own token.

3. The Fee Circuit

An off-chain keeper ticks every 15 seconds. Each tick, per token, it claims accrued fees from the locker when they exceed a \$0.50 dust floor, burns any token balance held by the treasury, routes the quote side, runs the buyback of Section 5, and marks the perpetual to market to decide among deposit, top-up, and take-profit.

The quote side of every claim is allocated by a split vector β over (perp, creator, protocol, buyback). The system ships with

$$\beta = (1, 0, 0, 0)$$

that is: 100% of collected fees become perpetual collateral. The protocol takes no share of fee flow at all; its only revenue is the profit share of Section 5. The creator’s share is likewise zero by default — the creator’s return is the token itself.

Deposits. The perpetual opens once the fee reserve reaches a gate $G = \$20$, and every further \$20 of fees is deposited as a top-up. A deposit is a plain USDG transfer to a deterministic intent address, which credits the token’s own Lighter account across the bridge. Positions run isolated margin. A deposit of collateral c at leverage $\lambda \in \{2, 3, 5, 10, 20\}$ extends the position by notional

$$N = \lambda \cdot c$$

executed as a market order with a 2% slippage cap.

Crash safety. The keeper’s internal buckets are checkpointed with write-ahead intent records, and every spend is decremented before the transaction is sent: a crash at any point under-pays a bucket for one tick rather than paying it twice. An ambiguous outcome — sent but unconfirmed — is promoted to a pending record and reconciled against onchain evidence; it is never blindly re-sent.

4. Tranche Accounting

The venue nets all of a token’s deposits into a single position. The keeper, however, keeps its own decomposition: every deposit creates a *tranche* (c_i, m_i, q_i) recording its collateral, its entry mark, and its size $q_i = \lambda c_i / m_i$.

Take-profit rule. A token’s risk profile, fixed at launch, is a trigger $r \in \{0.20, 0.50, 1.00\}$ (*safe, balanced, degen*). Tranche i banks when the mark reaches

$$m_i^* = m_i (1 + \sigma \frac{r}{\lambda})$$

where $\sigma = +1$ for a long and -1 for a short. Since the tranche’s profit at mark m is $q_i \sigma (m - m_i) = \lambda c_i \sigma (m - m_i) / m_i$, reaching m_i^* realizes exactly $r \cdot c_i$; a move of r/λ in the underlying is a return of $+r$ on that tranche’s own collateral.

No dilution from top-ups. Under averaged-entry accounting, a later deposit at a worse price shifts the aggregate break-even and postpones profit-taking on capital that was already in profit. Per-tranche targets are invariant to later deposits: every dollar of fees runs toward its own target from the price at which it entered. Figure 1 shows the required underlying move r/λ across the leverage range for the three risk profiles.

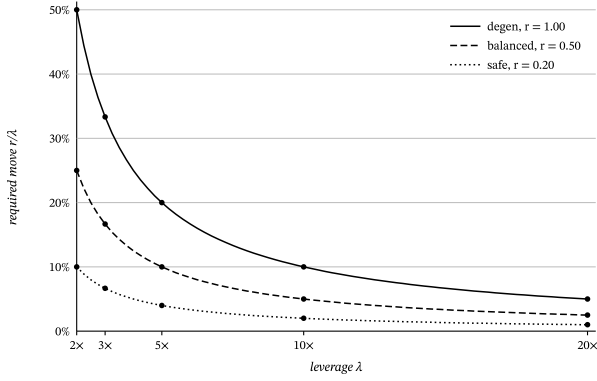


Figure 1: The underlying move r/λ required to bank a tranche, as a function of leverage, for the three risk profiles. Dots mark the leverages offered at launch. A 5× *safe* token banks each tranche on a 4% move in the underlying; a 2× *degen* token demands 50%.

Execution. When a tranche reaches its target, the keeper closes exactly that tranche’s size with a reduce-only immediate-or-cancel order under the 2% slippage cap, and credits realized profit only for the verified fill: the position is re-read after the order, and a partial fill leaves the remainder queued. Freed collateral re-enters as a fresh tranche at the current mark, so banked capital keeps compounding.

Reconciliation. The venue is authoritative. If the position is gone — liquidation — the tranche book zeroes. If the onchain size is smaller than the book, tranches are rescaled pro-rata; if larger, the excess becomes a synthetic tranche at the current entry. The keeper’s accounting can only ever converge toward what the venue reports.

Ring-fencing. Realized profit is excluded from the balance the keeper may redeploy (deployable = balance − realized). Profit destined for buybacks can never be re-risked on the position that produced it.

5. Profit Return and Supply Dynamics

Withdrawal. Once realized profit clears a floor $W = \$25$ — chosen to clear the venue’s minimum-withdrawal constraint with margin — it is withdrawn from Lighter over the zk bridge (roughly fifteen minutes in practice) back to the token’s treasury. The keeper recognizes the settlement from the onchain transfer itself and splits it: 25% to the operator treasury, 75% to the token’s buyback reserve.

Buyback. When a token’s buyback reserve crosses \$25, the keeper buys the token on its own pool and burns what it bought. A single tick spends at most \$25, with a minimum output quoted from live pool state less 3%, so a large reserve drains as a sequence of ordinary buys rather than one block-moving order.

Supply identity. Reported burns aggregate three channels in one number, $B(t) = S - \text{supply}(t)$: the fee-side

burns enforced by the locker on every claim; the buyback burns funded by realized profit; and any voluntary transfers to the dead address. The first channel requires no operator at all.

Operator revenue. The operator’s revenue is 25% of realized perpetual profit and nothing else: no launch fee, no share of fee flow, no share of the pool. The operator is paid if and only if the engines are profitable.

6. Properties

Burn continuity without the operator

Because *collect* is permissionless and its destinations are immutable, the fee-side burn and the routing of quote fees to the treasury continue to function with the keeper offline, operated by anyone willing to pay gas. The deflationary floor of every token survives the operator.

Bounded engine failure

The perpetual runs isolated margin, so the worst case of the engine is the loss of its posted collateral. The pool, the locked liquidity, and holder balances are outside the blast radius of a liquidation, and the engine re-opens at the next \$20 of fees.

Constant opening valuation

$MC_0 = V_0$ for every launch, regardless of the market, direction, or leverage chosen. The curve’s shape adapts; the opening valuation does not.

Per-token isolation

Each token has a distinct treasury address and a distinct venue account, and the venue only accepts orders from an account’s owner. No token’s flows can touch another token’s position, and no creator holds a key to any of it.

Realized, not marked, exposure

The token’s spot price does not mechanically track the underlying. Exposure reaches holders only through realized flows: take-profits that become buybacks and burns. This is the price of the bounded-failure property above, and it is deliberate.

Verifiability

Every launch, lock, claim, burn, deposit, and buyback is a confirmed transaction on Robinhood Chain [2], and each token’s treasury is a single public address. Quote inflow equals locker claims plus perpetual withdrawals; outflow equals venue deposits plus buyback swaps, which equal onchain burns and venue position deltas. The accounting closes, publicly.

Trust model

The design is explicit about what is trustless and what is not. Trustless: the token's supply and the absence of admin authority; the permanence of the liquidity lock; the routing and burning of fees. Trusted: the keeper's execution of deposit, take-profit, and buyback policy; custody of the master secret from which treasuries derive; the operator's 25% profit share. The operator can stall the engine; the operator cannot unlock liquidity or redirect fees, because the locker has no function for either.

7. Risks

Liquidation

An isolated position at leverage λ is liquidated by an adverse move on the order of $1/\lambda$. At 20 $\times$ that is a few percent. Liquidation forfeits the position's posted collateral and zeroes the tranche book; the engine restarts from the next \$20 of fees, and the contract-level fee burn is unaffected throughout. Higher leverage produces faster burns on a good call and faster liquidation on a bad one.

Operator dependency

Deposits, take-profits, withdrawals, and buybacks are executed by a single off-chain keeper. If it halts, those flows pause — fees keep accruing and the fee burn keeps running — until it resumes. More severely, treasury keys derive from the operator's master secret: its loss would strand treasury balances and venue collateral, though never the locked liquidity or holder balances.

Venue and bridge dependency

Lighter, its zk bridge, and the chain's RPC infrastructure are live dependencies. Outages pause deposits, closes, or withdrawals until they recover; bridge settlement adds latency (minutes, not blocks) to every profit return.

Early liquidity depth

A token opens at a \$4,000 valuation with one-sided liquidity, so early trades have high price impact in both directions. The keeper's own buybacks are bounded by the \$25 per-tick cap and the 3% slippage floor precisely so that the engine never trades meaningfully against that thinness.

Indirect exposure only

A buyer seeking continuous delta exposure to an underlying should hold the perpetual itself. The token delivers that exposure only as realized flows, on the keeper's schedule, and a position that never reaches its targets delivers none of it.

8. Conclusion

This paper specifies a launchpad in which a token's trading fees are not income to be split but collateral to be deployed. The token is inert and ownerless; its liquidity is locked in a contract that can only burn and forward; its fees run a live leveraged perpetual whose realized profits return as buybacks and burns. What is enforced by contract — supply, lock, burn — survives everyone, including the operator. What is operated — the engine — is bound in what it can lose and public in everything it does.

References

- [1] Lighter, "Lighter Documentation," <https://docs.lighter.xyz>
- [2] Robinhood Chain explorer (Blockscout), <https://robinhoodchain.blockscout.com>
- [3] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, D. Robinson, "Uniswap v3 Core," March 2021.
- [4] Global Dollar Network, "USDG — Global Dollar," <https://globaldollar.com>
- [5] H. Krawczyk, M. Bellare, R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, February 1997.